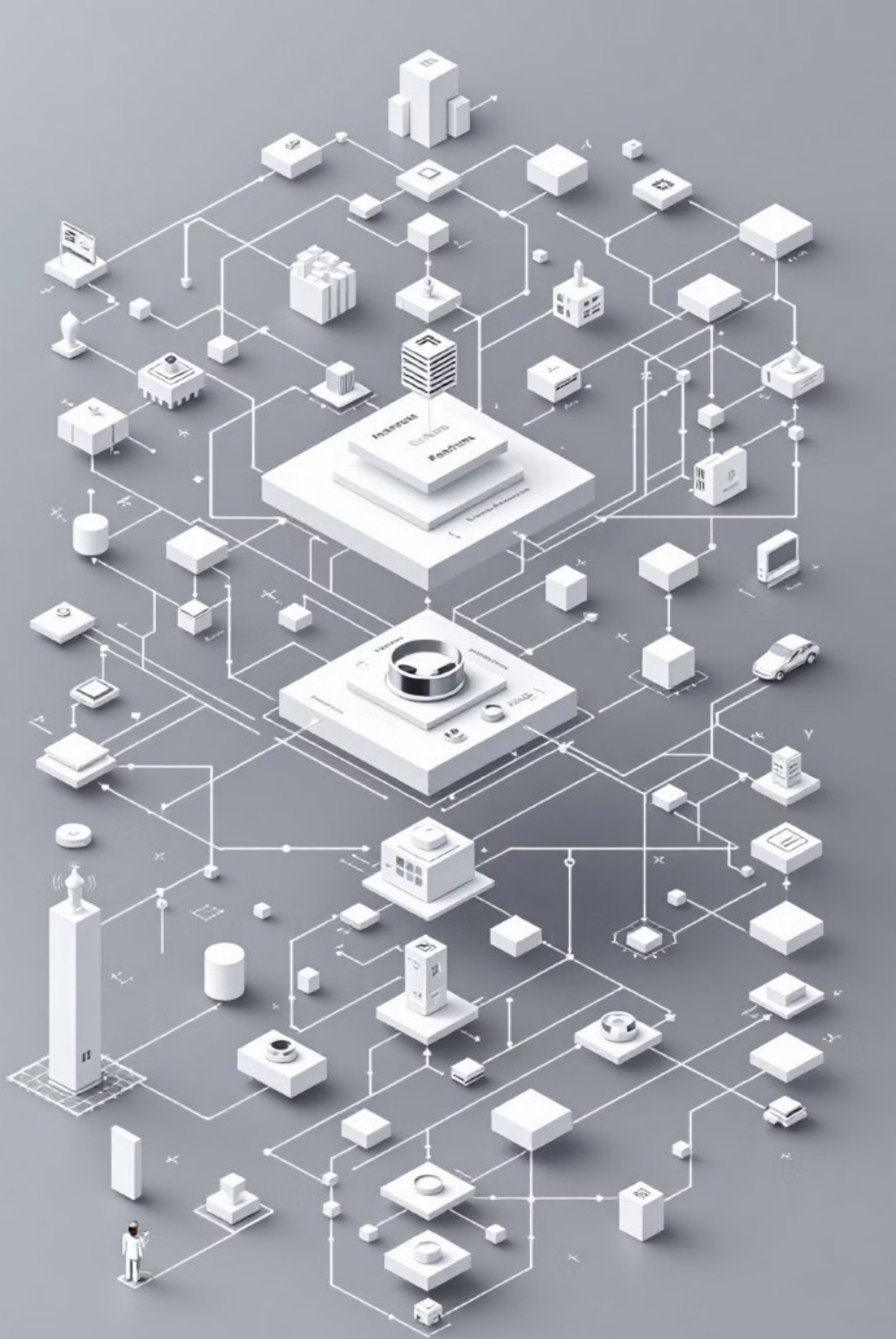




Arquitetura de Aplicação

Esta apresentação aborda a arquitetura de aplicações, focando em Servlets e IoT. Prepare-se para uma jornada informativa e técnica.

Jose Neto

Arquitetura Servlet

Servlets

Uma tecnologia Java para adicionar conteúdo dinâmico a servidores web.

API

Fornece uma interface para o servidor web através de uma API Java.

Conteúdo Dinâmico

Gera conteúdo dinâmico, geralmente HTML, interagindo com clientes via requisição-resposta.

javax.servlet

Servlet

1 API Java Servlet

Do pacote **javax.servlet**.

3 Modelo Requisição-Resposta

Interage com clientes usando este modelo.

2 Conteúdo Dinâmico

Adiciona conteúdo dinâmico em um servidor web.

4 Container Web

Necessita de um para ser executado.

Servlet



1.5emption
Exphase curdey
incround of lettrand in serverys

2.5reveys
Appustional forme of f serverys

Servlet

Se o método service for declarado, os demais métodos não serão executados.

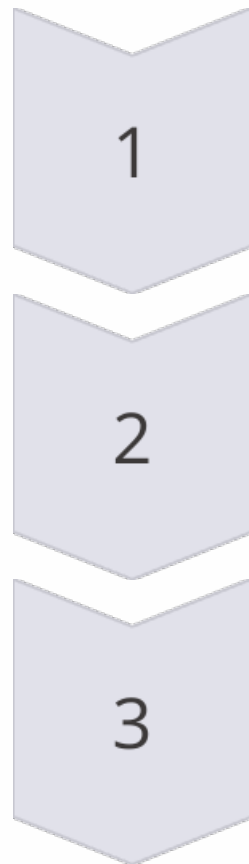
Método Service

Prioridade sobre outros métodos HTTP.



Servlet

Estrutura do Servlet



Request

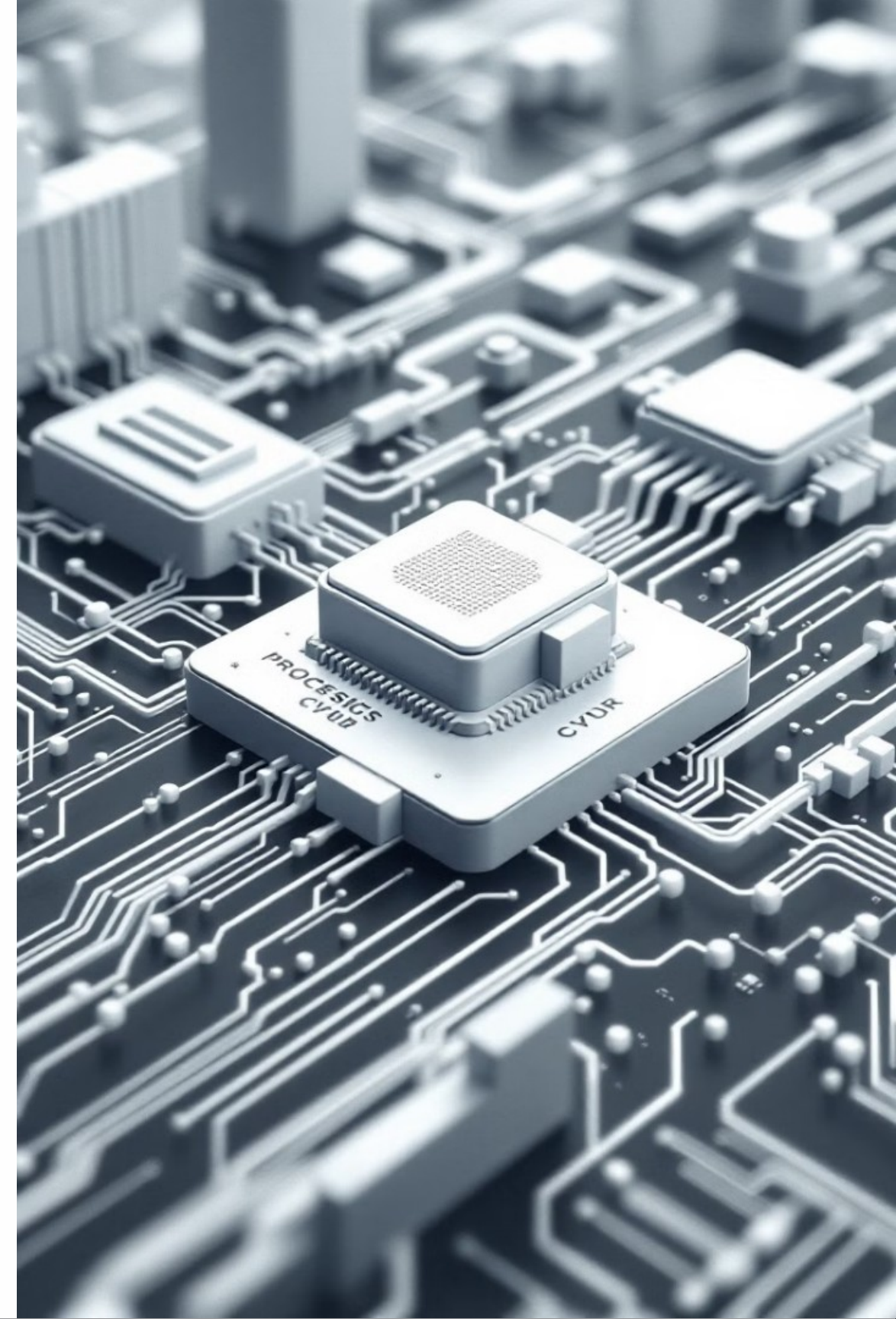
Requisição do cliente.

Servlet

Processamento da requisição.

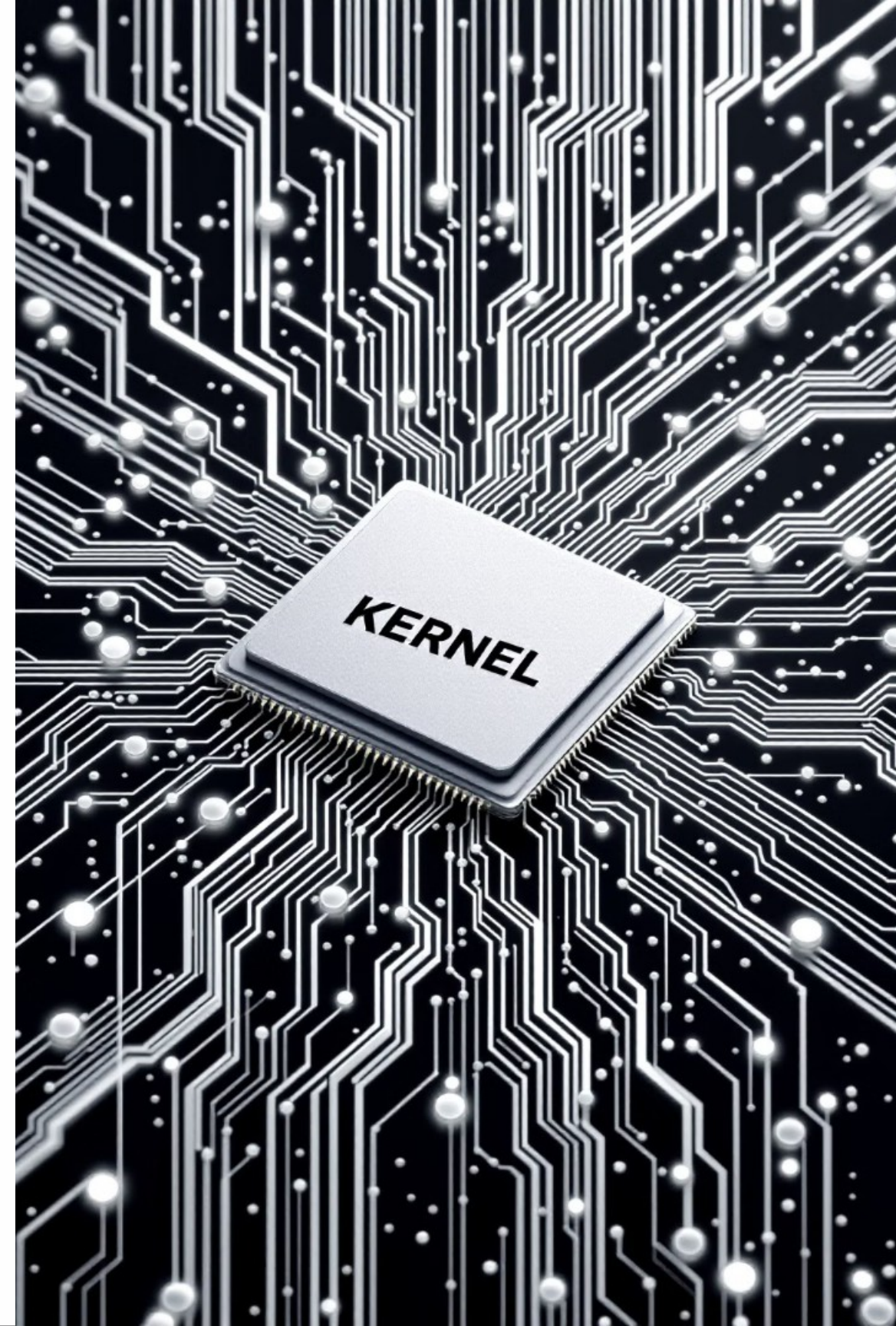
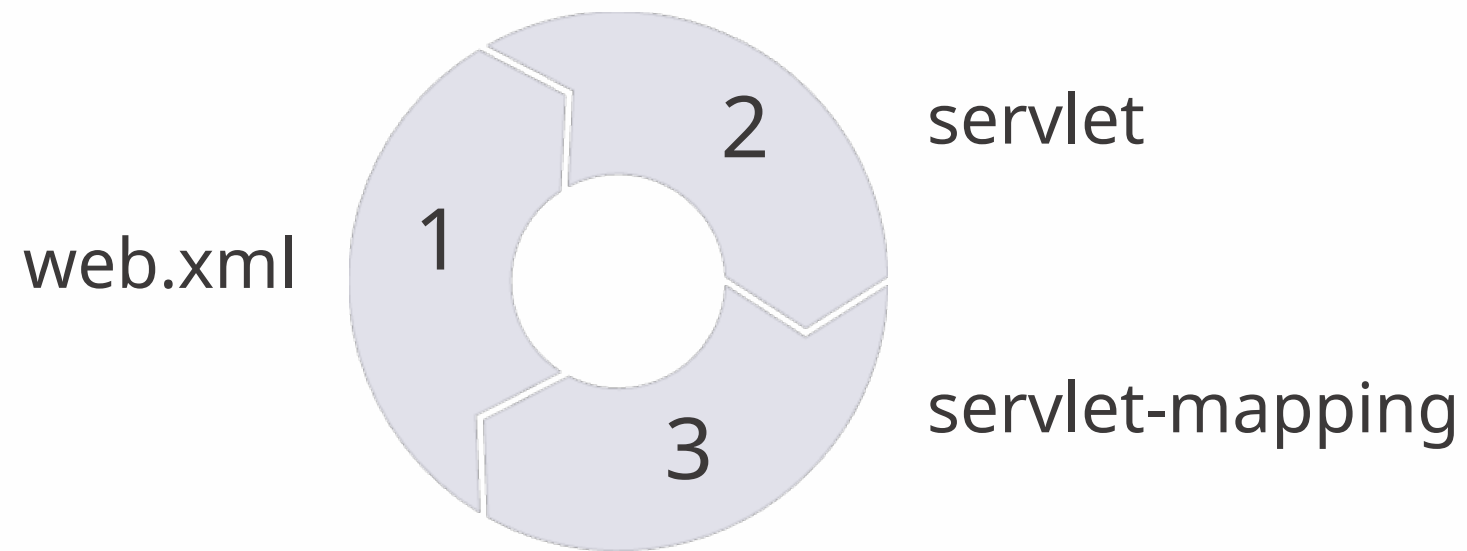
Response

Resposta ao cliente.

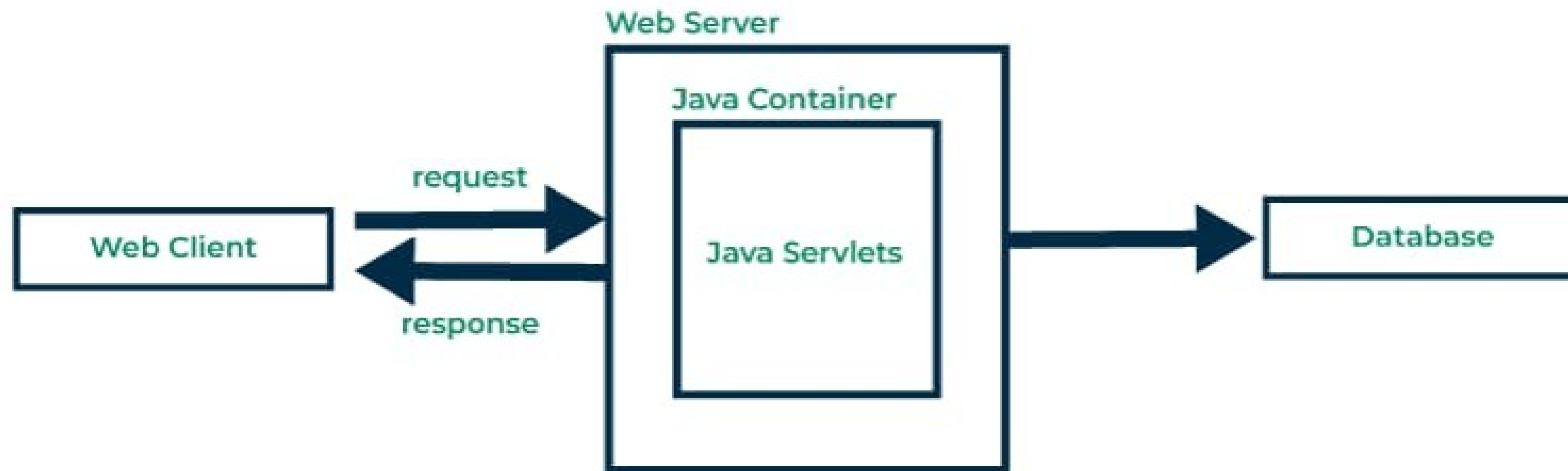


Servlet

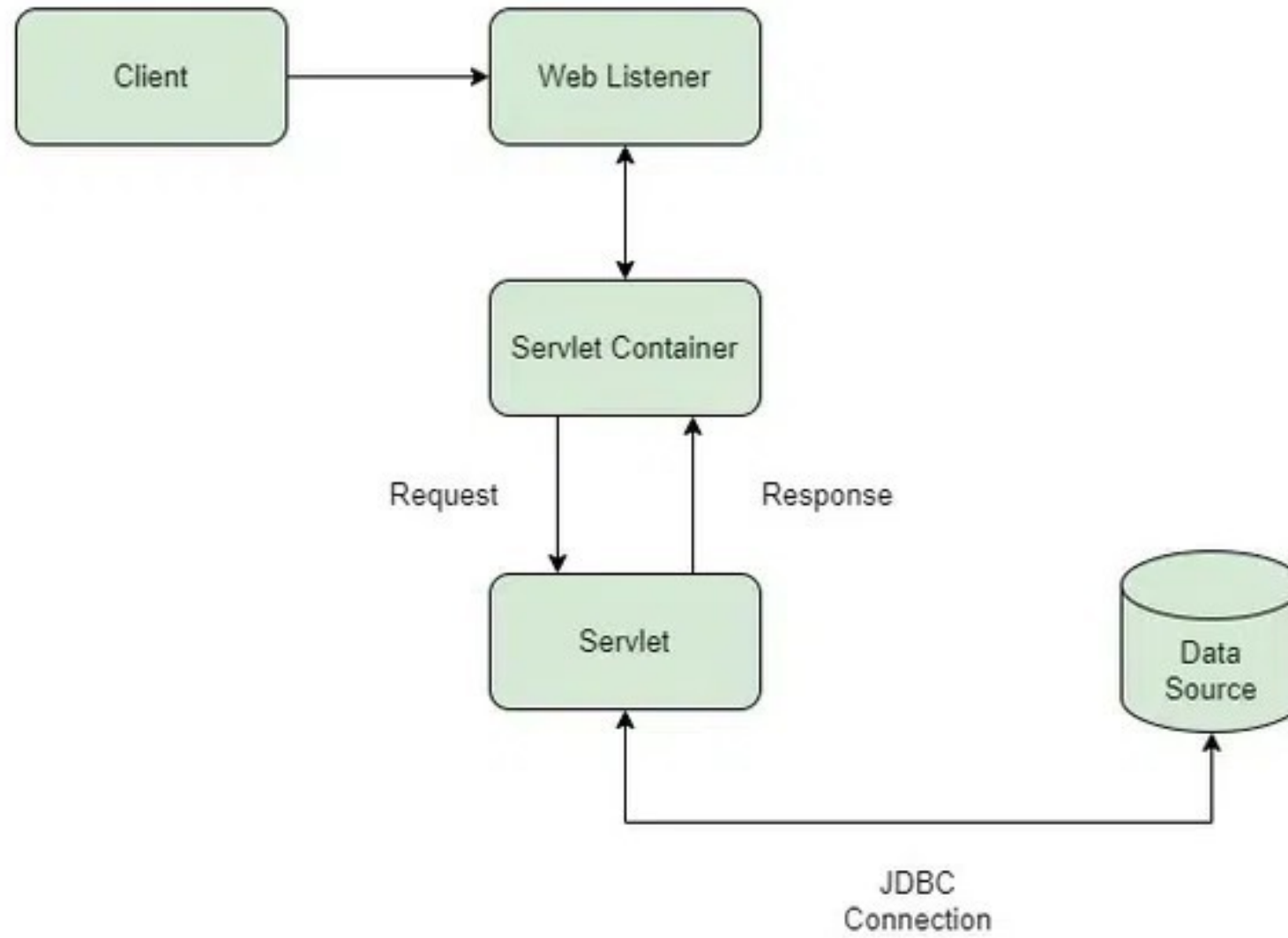
Todos os servlets criados na versão 2.5 ou inferior necessitam estar declarados no arquivo **web.xml** por meio de duas tags xml (servlet e servlet-mapping).



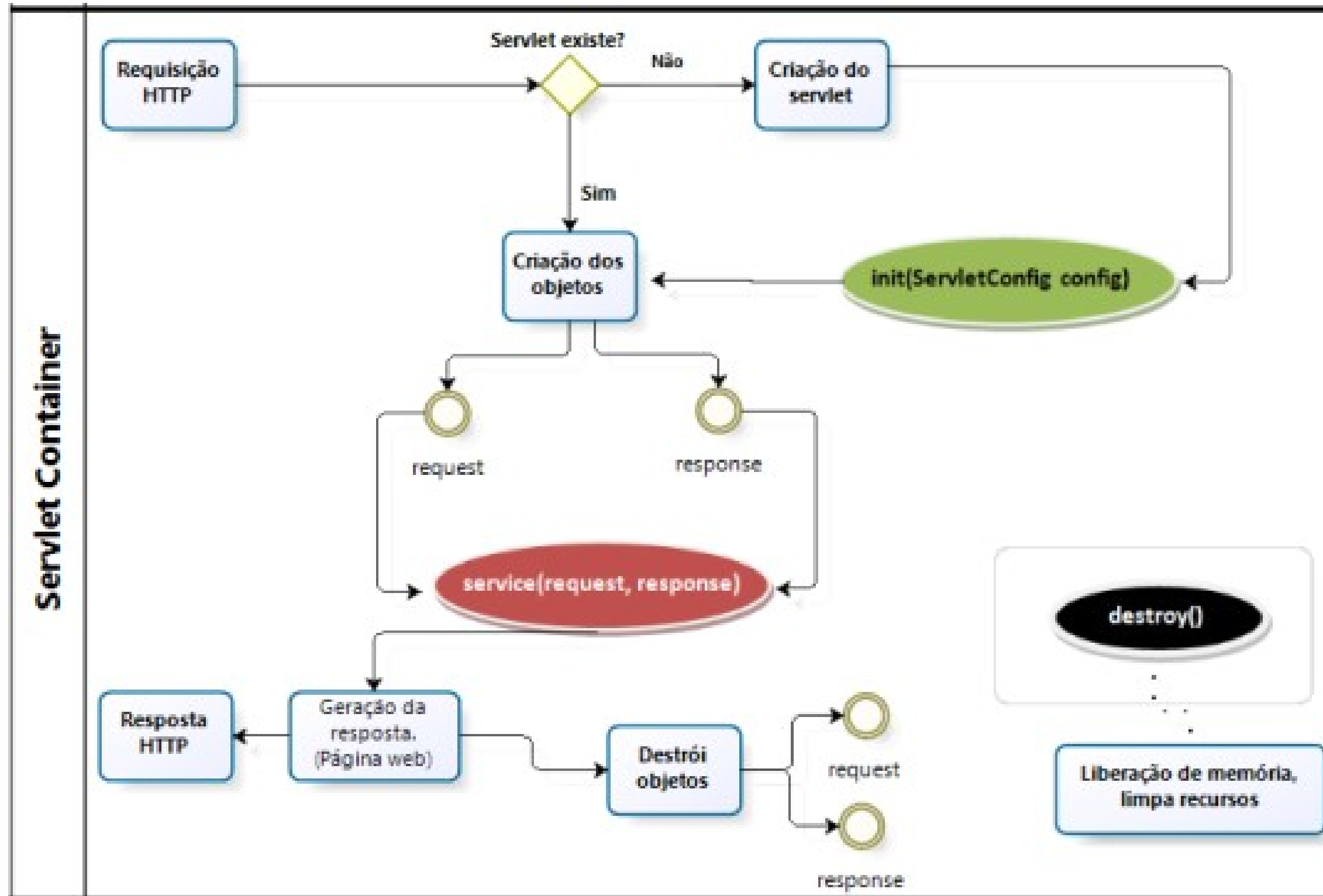
Servlet



Servlet



Servlet



javax.servlet

- Configuração do Servlet (Servlet 2.5 ou menor)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  <display-name>projeto-jsp</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>

  <servlet>
    <servlet-name>Index</servlet-name>
    <servlet-class>br.com.fiap.projeto.Index</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>Index</servlet-name>
    <url-pattern>/Index</url-pattern>
  </servlet-mapping>

</web-app>
```

Todos os servlets criados na versão 2.5 ou inferior necessitam estar declarados no arquivo **web.xml** por meio de duas tags xml (servlet e servlet-mapping).

- Configuração do Servlet (Servlet 3.0 ou superior)

```
import javax.servlet.annotation.WebServlet;  
  
@WebServlet("/Index2")  
public class Index2 extends HttpServlet {
```

import javax.servlet.annotation.WebServlet;

Importa a anotação @WebServlet, que é usada para mapear a URL que o servlet vai atender. Substitui a necessidade de configurar esse mapeamento no web.xml (mais antigo e verboso).

@WebServlet("/Index2")

Essa anotação mapeia o servlet para o caminho /Index2.

Isso quer dizer que se você acessar <http://localhost:8080/suaApp/Index2>, o servlet Index2 será executado.

Servlet

Construtor da Classe HttpServlet e serve para inicializar o objeto, ou seja, configurar os valores iniciais ou preparar o ambiente para que o objeto funcione corretamente.

```
public class Index extends HttpServlet {
```

```
    public Index() {  
        super();  
    }
```

Método que é chamado uma vez quando o servlet é carregado pela primeira vez.

```
    public void init(ServletConfig config) throws ServletException {  
        System.out.println("Init");  
    }
```

Método chamado quando o servlet está sendo destruído. Utilizado para liberar recursos.

```
    public void destroy() {  
        System.out.println("destroy");  
    }
```

Trata requisições HTTP Get pelo cliente

```
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException {  
        System.out.println("doGet");  
        response.getWriter().append("Served at: ").append(request.getContextPath());  
    }
```

getWriter() retorna um writer para escrever na resposta.

append("Served at: ") escreve esse texto.

append(request.getContextPath()) adiciona o contexto da aplicação web, como /minhaApp

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException {  
        System.out.println("doPost");  
        doGet(request, response);  
    }
```

Trata o POST da mesma forma que o GET.

Arquitetura de App

HORA DE DESENHAR



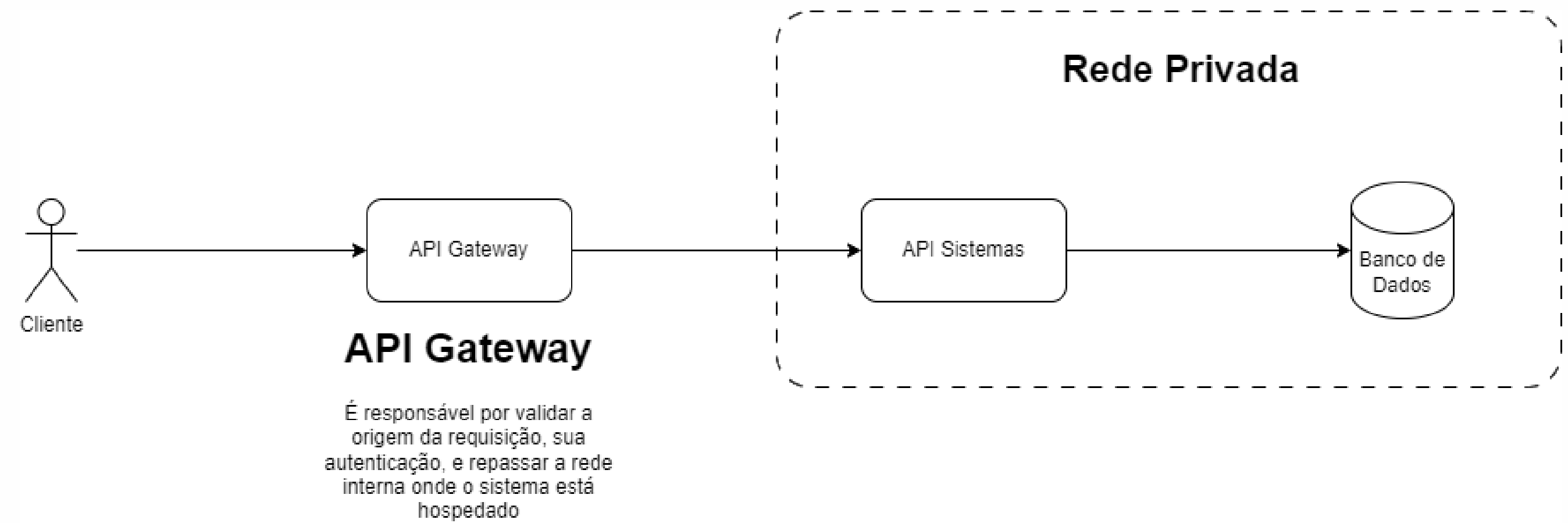
Diagrama esquemático.



Componentes da aplicação.



Arquitetura de App

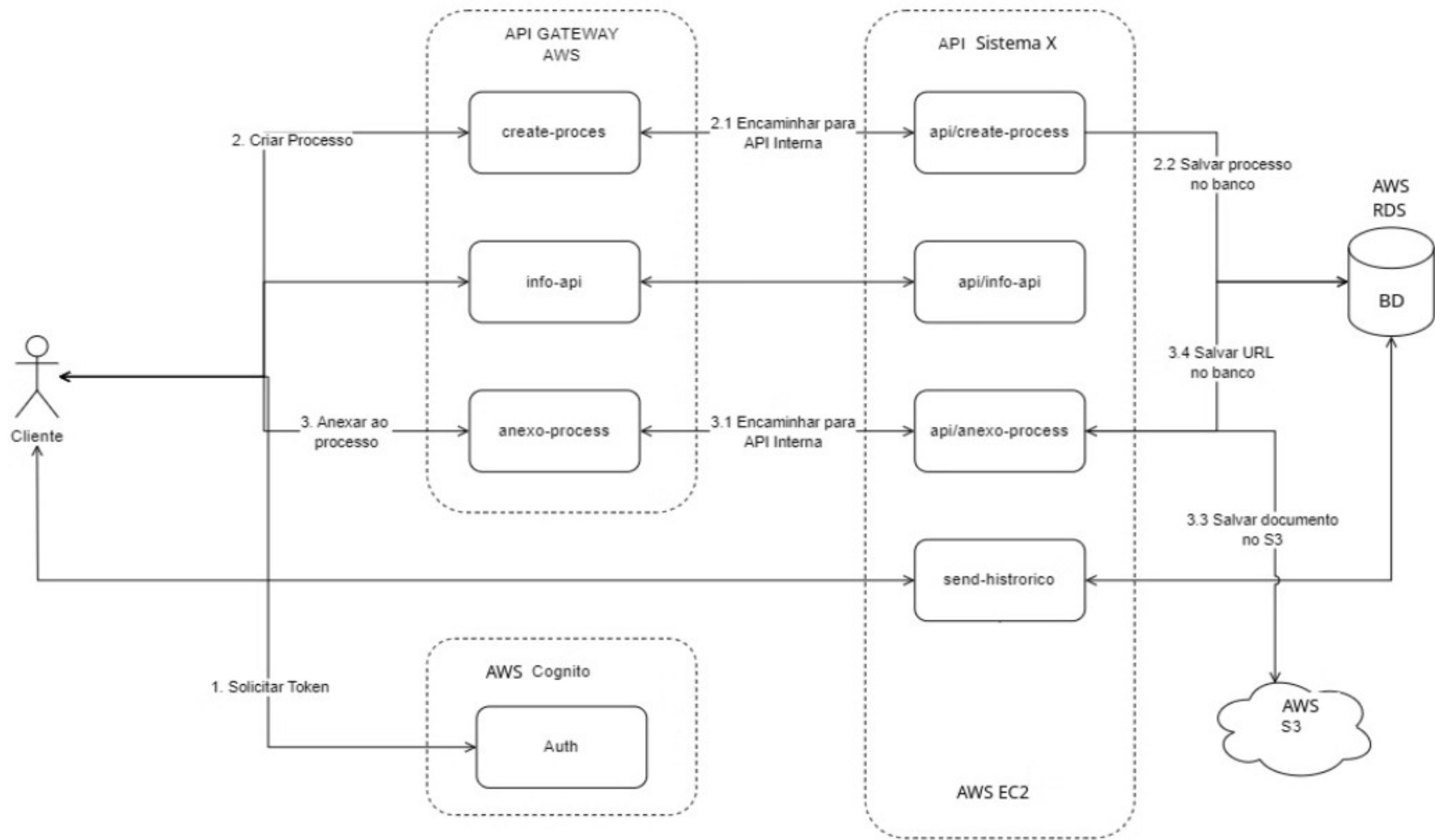


Hora de Desenhar

Arquitetura de App

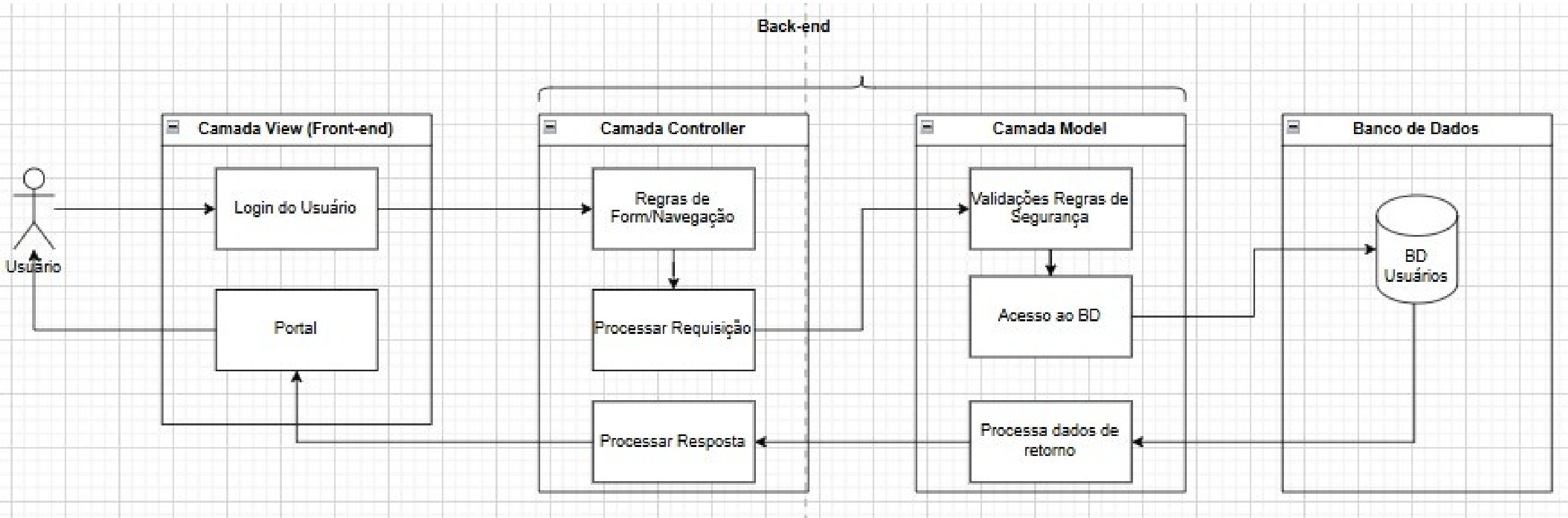
Detalhes
Visão detalhada da arquitetura.

Implementação
Passos para a implementação.



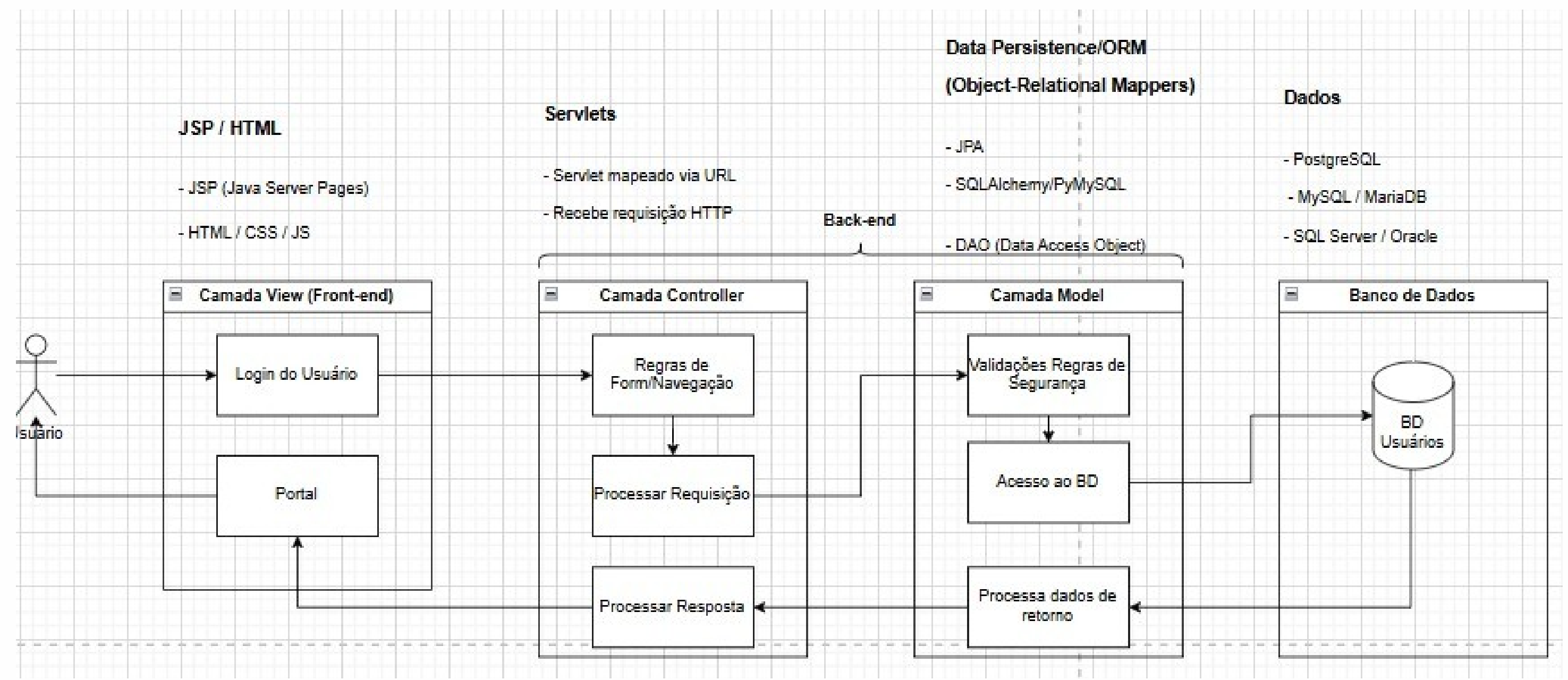
Hora de Desenhar

Arquitetura de App / MVC



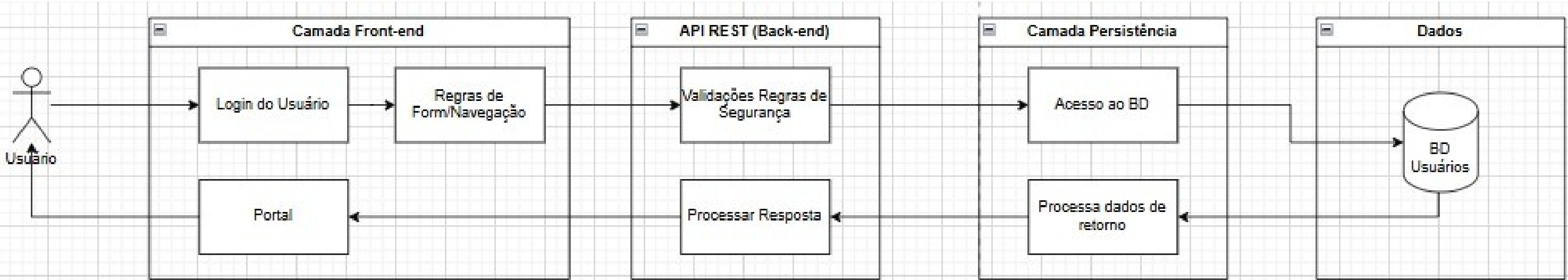
Hora de Desenhar

Arquitetura de App / Java Servlet MVC



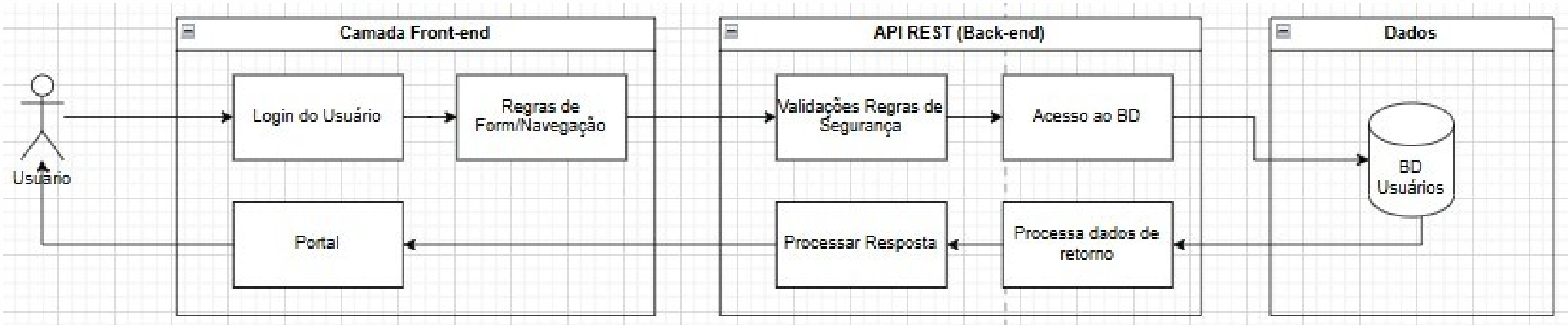
Hora de Desenhar

Arquitetura de App / REST



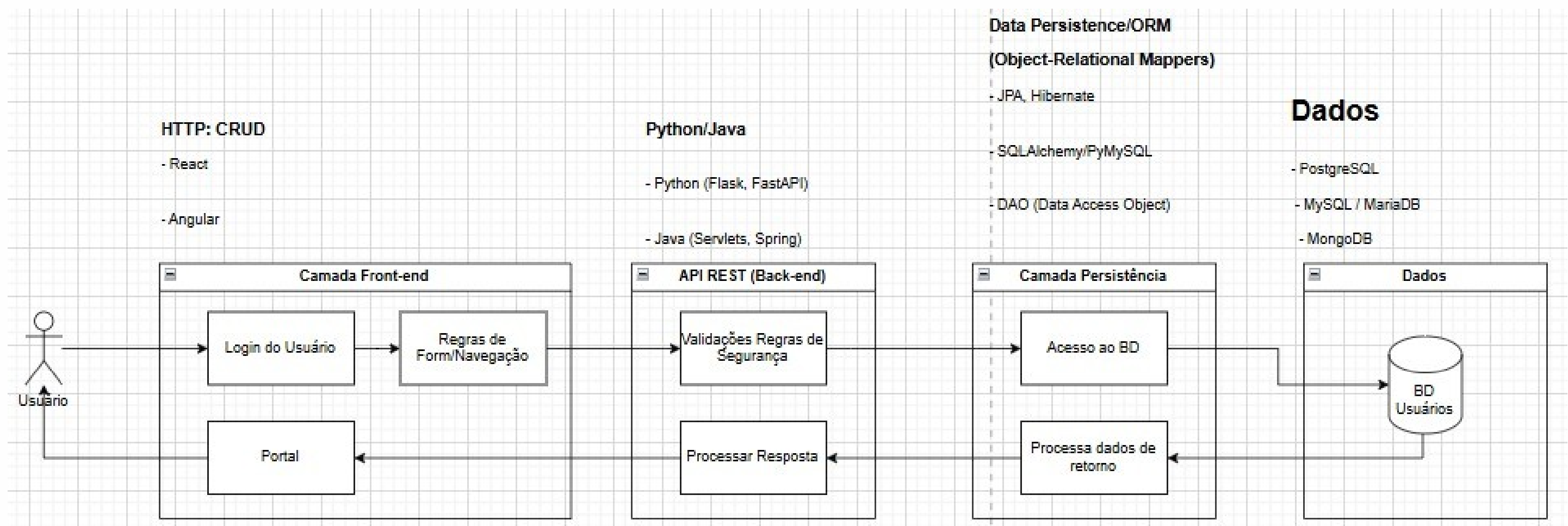
Hora de Desenhar

Arquitetura de App / REST



Hora de Desenhar

Arquitetura de App / REST



Hora de Desenhar

Arquitetura de App – Requisitos de Software

Escopo Geral

1. Agendamento de Consultas
2. Registro de Paciente
3. Gestão de Prontuários
4. Controle de Estoque de Medicamentos
5. Faturamento e Cobrança

Arquitetura de App – Roadmap



Arquitetura de App – Requisitos de Software

Histórias de Usuário

1. Agendamento de Consultas

Como um paciente,
Eu quero agendar, reprogramar ou cancelar consultas através de uma interface online,
Para que eu possa gerenciar minhas consultas de maneira conveniente, recebendo notificações de confirmações e lembretes.

2. Registro de Paciente

Como um novo paciente,
Eu quero me cadastrar no sistema fornecendo informações pessoais e histórico médico,
Para que meus dados sejam armazenados com segurança e estejam disponíveis para consultas futuras.

3. Gestão de Prontuários

Como um médico ou enfermeiro,
Eu quero acessar os prontuários eletrônicos dos pacientes,
Para que eu possa visualizar e atualizar informações de saúde, exames e prescrições durante o atendimento.

4. Controle de Estoque de Medicamentos

Como um administrador de farmácia hospitalar,
Eu quero gerenciar o estoque de medicamentos e materiais médicos,
Para que eu possa monitorar o reabastecimento e garantir que os produtos estejam dentro da validade.

5. Faturamento e Cobrança

Como um administrador hospitalar,
Eu quero gerar faturas para procedimentos realizados,
Para que eu possa registrar pagamentos e convênios de saúde, garantindo a precisão nas cobranças e a gestão financeira.

Arquitetura de App

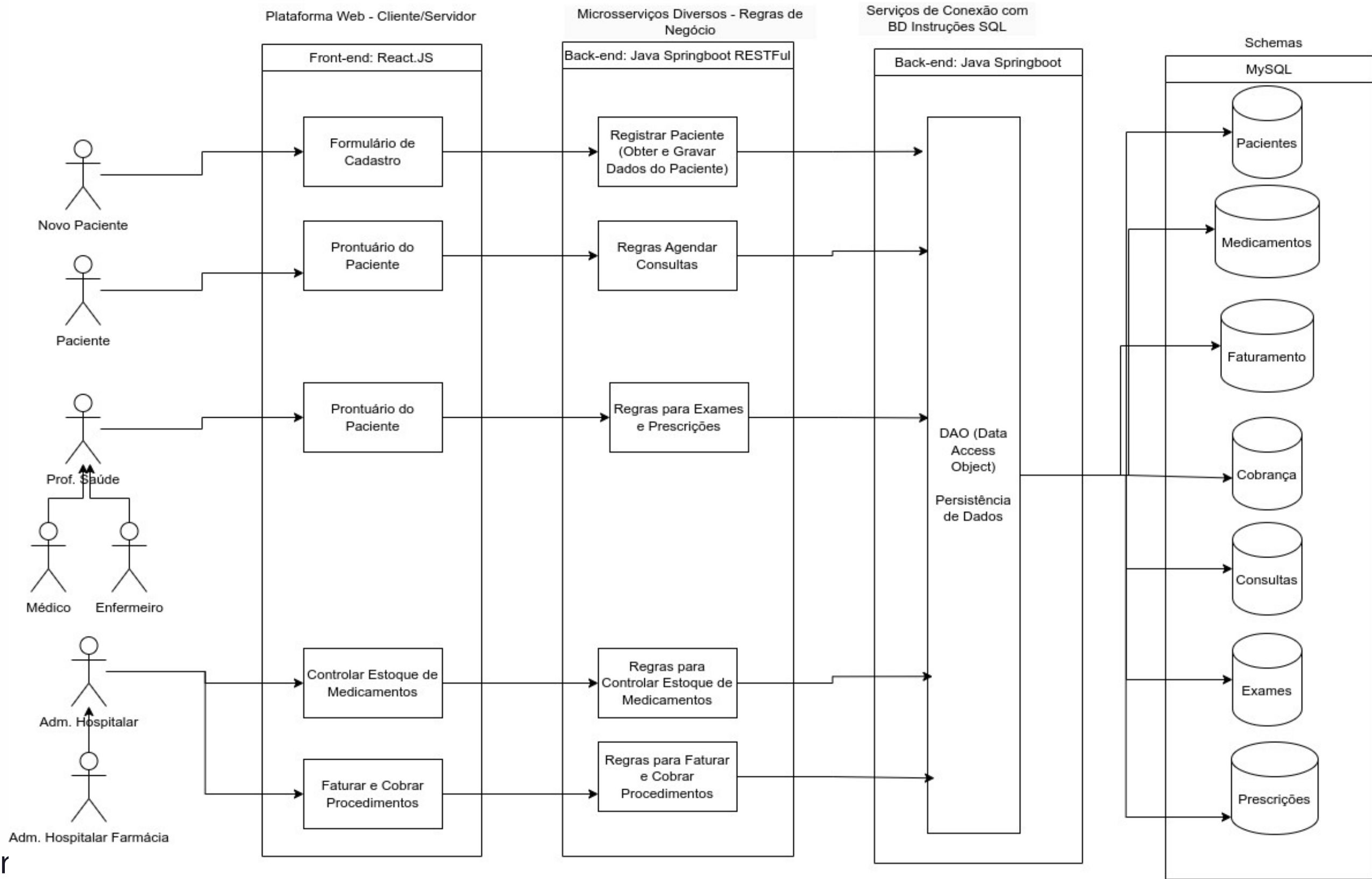
Detalhes

Visão detalhada da arquitetura.

Implementação

Passos para a implementação.

Hora de Desenhar





Internet das Coisas (IoT)

Internet of Things (IoT)

Explorando o mundo da IoT e suas aplicações na arquitetura moderna.

Conectividade

Dispositivos interconectados.

Cidades Inteligentes (SC)

Smart Cities (SC)

IoT – Internet of Things

A Conexão Inteligente de Dispositivos no Mundo Moderno

O que é IoT?

Definição: A Internet das Coisas (IoT) é uma rede de objetos físicos ('coisas') que estão equipados com sensores, software e outras tecnologias com o objetivo de conectar e trocar dados com outros dispositivos e sistemas pela Internet.

Exemplos: lâmpadas, termostatos, câmeras de segurança, carros com computação móvel e embarcada, etc.



Mais Exemplos:

- Eletrodomésticos,
- Câmeras de segurança,
- Semáforos Inteligentes,
- Sensores ambientais,
- Dispositivos médicos
- Lixeiras Inteligentes



Conceito de Cidades Inteligentes (Smart Cities)

Cidade Inteligente (Smart City) é um ambiente urbano que usa tecnologia e análise de dados para:

1

Otimizar serviços públicos e reduzir impactos ambientais

2

Melhorar a qualidade de vida da população

3

Promover a sustentabilidade econômica e social.



A transformação em Cidades Inteligentes depende do uso eficiente de tecnologias como:

1

IoT (Internet das Coisas)

2

Big Data (grandes volumes de dados)

3

IA (Inteligência Artificial)

4

Integração de tudo isso!

IoT

Como Funciona a IoT?

Componentes Principais:

1

Sensores

Coletam dados.

2

Conectividade

Transmitem dados.

3

Processamento

Analisa dados.

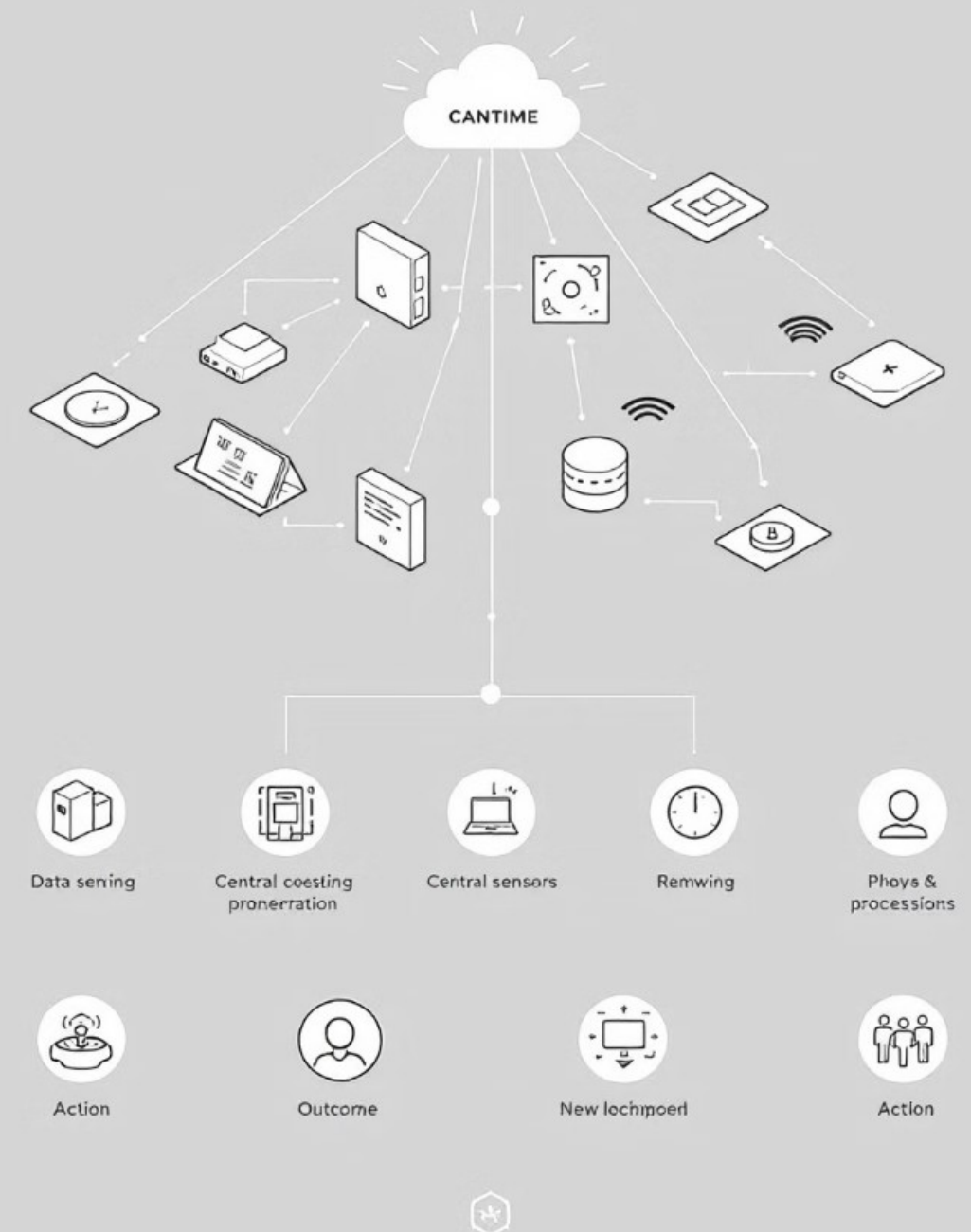
4

Ação

Dispositivos agem na tarefa.

IOT WORKFLOW

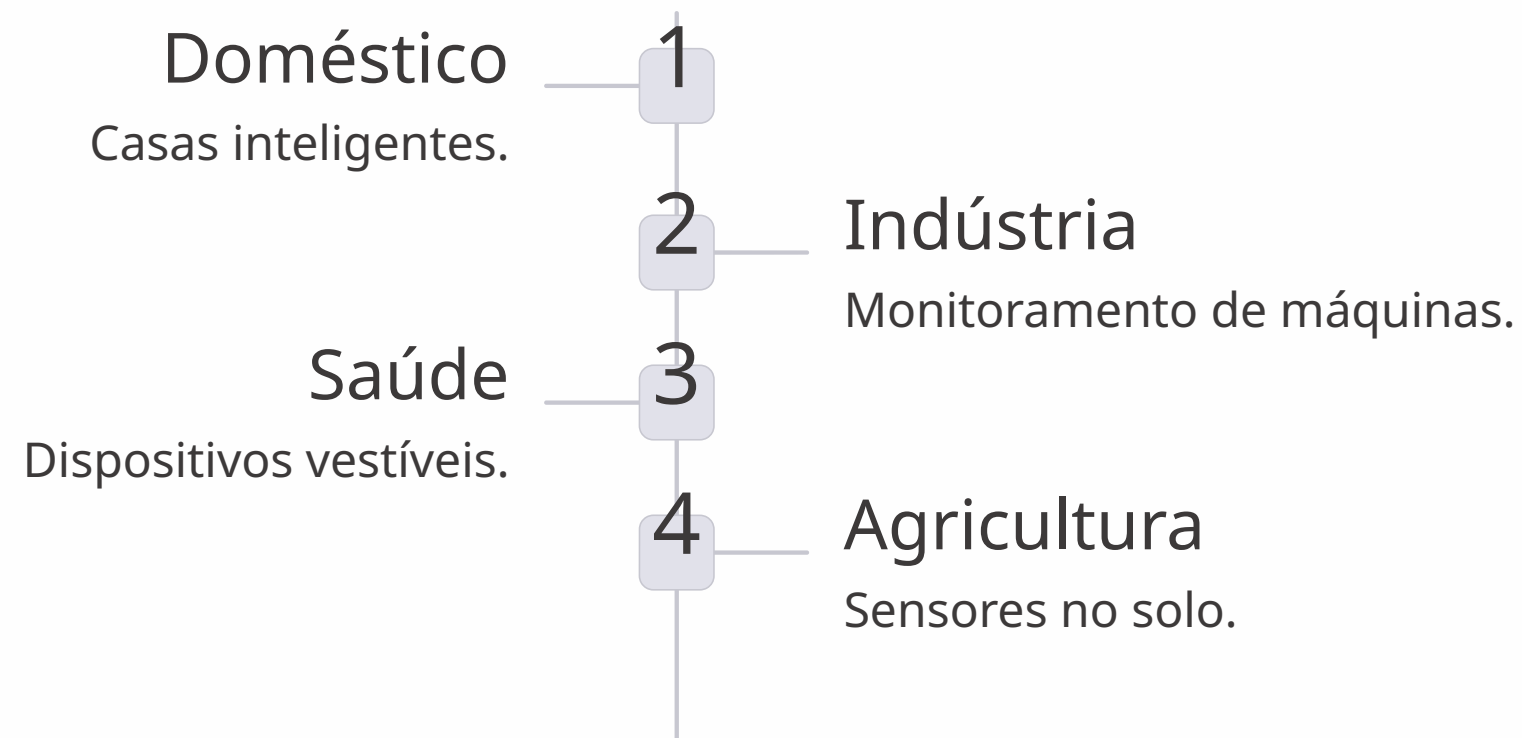
The easy calculation of the the moder atess nake whilangr adicition data on delectation interation a line your make dly dlog a ncourior icaa been our cononhing contrats connden of the prntess on and pechusioe for recloww cliacone any protecting mers.





IoT

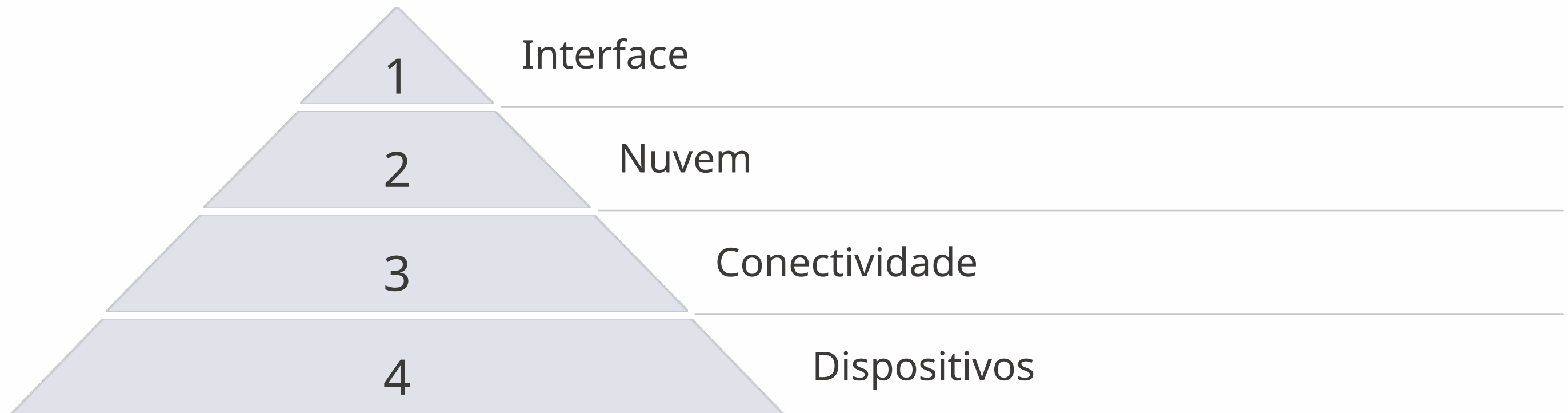
Exemplos de Uso da IoT



IoT

Arquitetura IoT

Camadas da Arquitetura IoT:



Cada camada desempenha um papel vital.

Importante: A arquitetura é orientada à eficiência no uso de recursos e respostas rápidas aos dados capturados.

Arquitetura de IoT

A arquitetura típica de IoT é organizada em camadas funcionais:

1 Camada de Percepção

Função: Captura dados do ambiente físico.

Dispositivos: Sensores (temperatura, luminosidade, poluição, umidade) e atuadores (motores, lâmpadas inteligentes).

2 Camada de Processamento

Função: Armazena, filtra, processa e analisa os dados coletados.

Componentes: Computadores locais, servidores de borda (edge computing), e serviços de nuvem.

3 Camada de Aplicação

Função: Apresenta serviços úteis aos usuários ou sistemas, como aplicativos móveis, sistemas de gerenciamento urbano e automações.

Arquitetura de Cidades Inteligentes

Uma Cidade Inteligente baseia-se em um conjunto de sistemas integrados:

1 Sensoriamento Urbano

Sensores distribuídos pela cidade monitoram mobilidade urbana, qualidade do ar, consumo energético e segurança pública.

2 Plataformas de Dados

Centros de processamento que armazenam e analisam os dados para transformar informação em inteligência estratégica.

3 Centros de Controle Operacional (CCO)

Estruturas físicas equipadas com monitores e softwares que acompanham, em tempo real, a dinâmica da cidade (trânsito, emergências, eventos climáticos).

IoT

Desafios e Considerações de Segurança:

Camadas da Arquitetura IoT:

1 Segurança de Dados

Vulnerabilidades em dispositivos.

2 Privacidade

Proteção de dados pessoais.

3 Integração

Comunicação eficiente.



IoT - Tecnologias de Apoio

Para que IoT e cidades inteligentes funcionem de forma eficiente, utilizam-se tecnologias como:

- 1 Edge Computing (Computação na Borda)**
Processamento de dados próximo da fonte para reduzir latência.
- 2 Big Data (Grandes volumes de dados)**
Coleta e análise de dados em larga escala para tomada de decisão.
- 3 IA (Inteligência Artificial)**
Capacidade dos sistemas de identificar padrões, prever eventos e automatizar ações.

IoT/SC - Exemplos Reais e Atuais de Aplicação

1 Transporte Inteligente

Exemplo: Waze for Cities (Google) integra dados de trânsito em tempo real com administrações municipais.

Aplicação real: São Paulo (SP) usa informações do Waze para reprogramar semáforos e rotas em eventos.

2 Iluminação Pública Inteligente

Exemplo: Postes inteligentes com sensores de movimento e sensores de luminosidade.

Aplicação real: Barcelona (Espanha) reduz consumo energético com ajuste automático da intensidade da luz.

3 Gestão Inteligente de Resíduos

Exemplo: Contentores inteligentes equipados com sensores de enchimento.

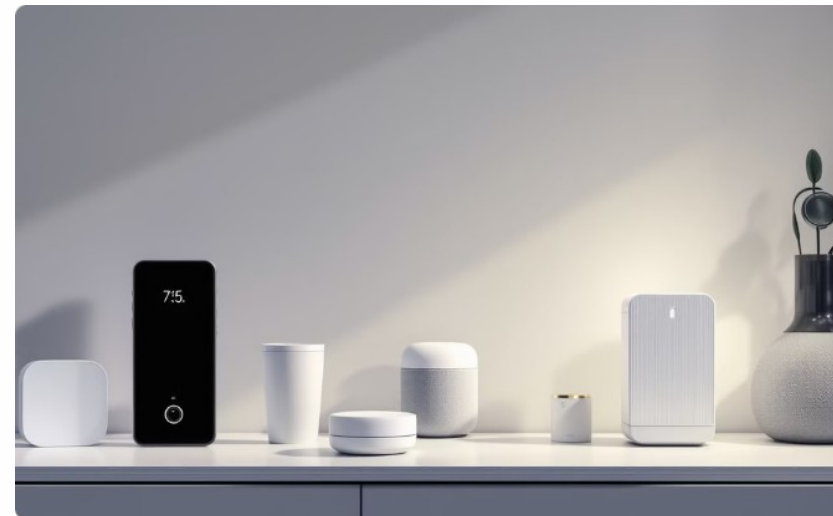
Aplicação real: Dublin (Irlanda) conseguiu diminuir o número de viagens de coleta em até 85%, economizando combustível e reduzindo emissão de carbono.

IoT/SC - Exemplos Reais e Atuais de Aplicação

1 Estacionamento Inteligente

Exemplo: Sensores de vaga que informam a disponibilidade em tempo real via aplicativos.

Aplicação real: San Francisco (EUA) usa o sistema SFpark para reduzir congestionamentos e melhorar o fluxo de veículos.

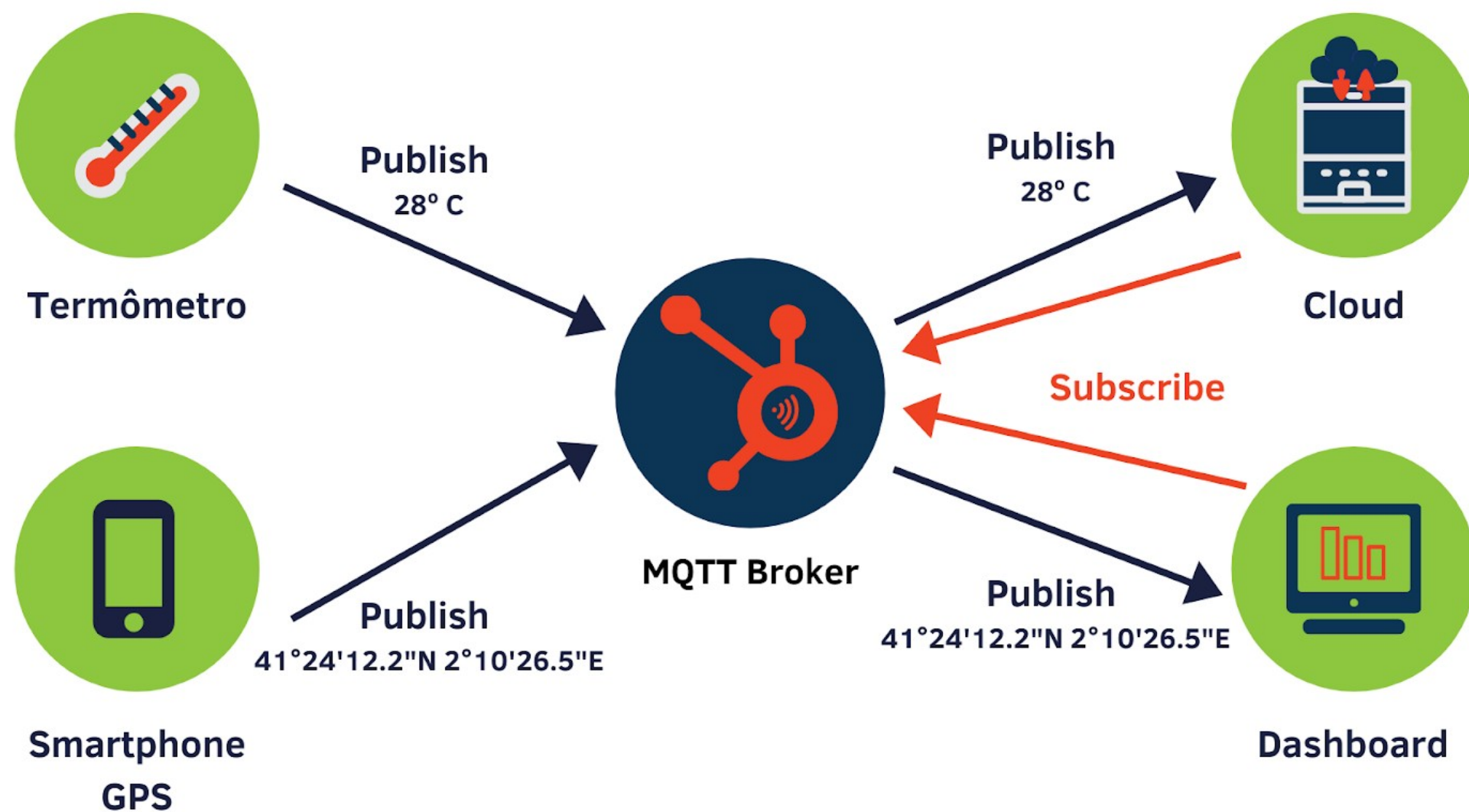


2 Monitoramento Ambiental

Exemplo: Estações de sensores de qualidade do ar e ruído espalhadas pela cidade.

Aplicação real: Curitiba (PR) monitora condições ambientais em tempo real para desenvolver políticas públicas de sustentabilidade.

IoT/SC - Exemplos Reais e Atuais de Aplicação



MQTT – Message Queuing Telemetry Transport: Protocolo de transporte de mensagem leve que é ideal para dispositivos com recursos limitados.



Desafios para Implementação

1

Privacidade e segurança de dados:

Garantir que as informações coletadas sejam protegidas contra acessos não autorizados.

2

Interoperabilidade entre dispositivos:

Integrar sistemas e equipamentos de diferentes fabricantes.

3

Manutenção e atualização dos dispositivos:

Atualizar sensores, plataformas e softwares periodicamente para garantir eficácia.

4

Inclusão digital:

Assegurar que toda a população possa usufruir dos benefícios das cidades inteligentes.



IoT

O Futuro da IoT

Tendências Futuras:

1

Cidades Inteligentes
Expansão da automação.

2

Carros Autônomos
Uso crescente da IoT.

3

Integração com IA
Automação avançada.

Arquitetura de App

HORA DE DESENHAR



Diagrama esquemático.



Componentes da aplicação.



Esboço do Projeto de IoT | Casa Inteligente

